Documento de Execução — Plataforma REVISA

1. Objetivo desta versão

Esta versão transforma a arquitetura conceitual em uma especificação de execução para orientar implementação final do monorepo, banco de dados, APIs, autorização e convenções de desenvolvimento.

O foco aqui é fechar quatro blocos executáveis:

- matriz detalhada de permissões por endpoint
- DDL SQL final por schema
- contratos OpenAPI por módulo
- estrutura exata do monorepo com convenções e responsabilidades

A diretriz central permanece:

- **monorepo único**
- **backend como monólito modular**
- **frontend web por áreas funcionais**
- **mobile dedicado à captação territorial**
- **codificação por módulos fechados de ponta a ponta**
- **sem incrementalismo excessivamente fragmentado**

---

2. Regras de implementação obrigatórias

2.1 Regra de fechamento funcional

Nenhum módulo será considerado pronto se entregar apenas banco, apenas endpoint ou apenas tela.

Cada módulo deve fechar:

- DDL
- models
- schemas
- services
- repositories
- endpoints
- autorização
- auditoria
- testes mínimos
- tela correspondente quando aplicável

2.2 Regra de nomenclatura

- tabelas em `snake_case` e no plural, exceto tabelas técnicas de junção simples
- endpoints REST em `kebab-case`
- nomes de módulos em inglês técnico no código

- nomes de negócio podem aparecer em português apenas em labels, seeds e documentação
- enums centrais em pacote compartilhado

2.3 Regra de escopo

Toda leitura e mutação de dado de negócio deve passar por: 1. autenticação 2. checagem de permissão 3. checagem de escopo 4. auditoria quando aplicável

2.4 Regra de governança

Toda operação abaixo deve gerar log:

- login/logout
- falha de autenticação
- alteração de perfis
- alteração de escopos
- exportação
- visualização de dados sensíveis
- revogação de consentimento
- exclusão/anonimização

---

3. Matriz detalhada de permissões por endpoint

3.1 Convenções de autorização

Perfis principais

- `ADM_GERAL_REVISA`
- `ADM_REVISA`
- `VEREADOR`
- `CHEFE_GABINETE`
- `SUPERVISOR_EQUIPE_POLITICA`
- `ADM_POLO`
- `COORDENADOR_POLO`
- `COLABORADOR_POLO`
- `COLABORADOR_GABINETE`
- `COLABORADOR_REVISA`
- `BENEFICIARIO`
- `EMPRESA_PARCEIRA`
- `VOLUNTARIO_AUTOINSCRITO`

Permissões atômicas centrais

- `auth.login`
- `user.read`
- `user.create`
- `user.update`
- `user.manage_roles`
- `user.manage_scopes`

- `organization.read`
- `organization.create`
- `organization.update`
- `vereador.read`
- `vereador.create`
- `vereador.update`
- `team.read`
- `team.create`
- `team.update`
- `person.read`
- `person.create`
- `person.update`
- `person.link`
- `consent.read`
- `consent.create`
- `consent.revoke`
- `capture.read`
- `capture.create`
- `capture.classify`
- `capture.forward`
- `capture.convert`
- `polo.read`
- `polo.create`
- `polo.update`
- `polo.manage_beneficiary`
- `modality.read`
- `modality.create`
- `modality.update`
- `attendance.read`
- `attendance.create`
- `occurrence.read`
- `occurrence.create`
- `daily_log.read`
- `daily_log.create`
- `purchase_request.read`
- `purchase_request.create`
- `cabinet.read`
- `cabinet.action.read`
- `cabinet.action.create`
- `task.read`
- `task.create`
- `task.update`
- `task.complete`
- `demand.read`
- `demand.create`
- `demand.update`
- `demand.assign`
- `event.read`
- `event.create`
- `event.update`

- `dashboard.admin.read`
- `dashboard.vereador.read`
- `dashboard.polo.read`
- `dashboard.cabinet.read`
- `geo.read`
- `geo.manage`
- `report.read`
- `report.export`
- `audit.read`
- `privacy.read`
- `privacy.process`

Escopos

- `GLOBAL`
- `REVISA`
- `VEREADOR`
- `GABINETE`
- `POLO`
- `EQUIPE`
- `SELF`

3.2 Matriz por grupo de endpoints

3.2.1 Auth

Método	Endpoint	Permissão	Perfis permitidos	Escopo
POST	`/api/v1/auth/login`	pública	todos usuários ativos	n/a
POST	`/api/v1/auth/refresh`	token válido	todos usuários ativos	n/a
POST	`/api/v1/auth/logout`	autenticado	todos	SELF
POST	`/api/v1/auth/forgot-password`	pública	usuário conhecido	n/a
POST	`/api/v1/auth/reset-password`	token de reset	usuário alvo	SELF
GET	`/api/v1/auth/me`	autenticado	todos	SELF

3.2.2 Usuários e IAM

Método	Endpoint	Permissão	Perfis permitidos	Escopo
GET	`/api/v1/users`	`user.read`	ADM_GERAL_REVISA, ADM_REVISA	GLOBAL/ REVISA

Método	Endpoint	Permissão	Perfis permitidos	Escopo
POST	`/api/v1/users`	`user.create`	ADM_GERAL_REVISA, ADM_REVISA	GLOBAL/REVISA
GET	`/api/v1/users/{id}`	`user.read`	ADM_GERAL_REVISA, ADM_REVISA, próprio usuário	GLOBAL/REVISA/SELF
PATCH	`/api/v1/users/{id}`	`user.update`	ADM_GERAL_REVISA, ADM_REVISA, próprio usuário limitado	GLOBAL/REVISA/SELF
POST	`/api/v1/users/{id}/roles`	`user.manage_roles`	ADM_GERAL_REVISA	GLOBAL
POST	`/api/v1/users/{id}/scopes`	`user.manage_scopes`	ADM_GERAL_REVISA, ADM_REVISA	GLOBAL/REVISA
POST	`/api/v1/users/{id}/block`	`user.update`	ADM_GERAL_REVISA, ADM_REVISA	GLOBAL/REVISA

3.2.3 Organizações, vereadores, equipes

Método	Endpoint	Permissão	Perfis permitidos	Escopo
GET	`/api/v1/organizations`	`organization.read`	ADM_GERAL_REVISA, ADM_REVISA	GLOBAL/REVISA
POST	`/api/v1/organizations`	`organization.create`	ADM_GERAL_REVISA	GLOBAL
PATCH	`/api/v1/organizations/{id}`	`organization.update`	ADM_GERAL_REVISA, ADM_REVISA	GLOBAL/REVISA
GET	`/api/v1/vereadores`	`vereador.read`	ADM_GERAL_REVISA, ADM_REVISA	GLOBAL/REVISA
POST	`/api/v1/vereadores`	`vereador.create`	ADM_GERAL_REVISA	GLOBAL
PATCH	`/api/v1/vereadores/{id}`	`vereador.update`	ADM_GERAL_REVISA, ADM_REVISA	GLOBAL/REVISA

Método	Endpoint	Permissão	Perfis permitidos	Escopo
GET	`/api/v1/teams`	`team.read`	ADM_GERAL_REVISA, ADM_REVISA, CHEFE_GABINETE, SUPERVISOR_EQUIPE_POLITICA, ADM_POLO, COORDENADOR_POLO	conforme vínculo
POST	`/api/v1/teams`	`team.create`	ADM_GERAL_REVISA, ADM_REVISA	REVISA
PATCH	`/api/v1/teams/{id}`	`team.update`	ADM_GERAL_REVISA, ADM_REVISA, CHEFE_GABINETE, ADM_POLO	conforme vínculo

3.2.4 Pessoas e consentimentos

Método	Endpoint	Permissão	Perfis permitidos	Escopo
GET	`/api/v1/persons`	`person.read`	ADM_GERAL_REVISA, ADM_REVISA, CHEFE_GABINETE, SUPERVISOR_EQUIPE_POLITICA, ADM_POLO, COORDENADOR_POLO, COLABORADOR_GABINETE, COLABORADOR_POLO, COLABORADOR_REVISA	conforme escopo
POST	`/api/v1/persons`	`person.create`	ADM_GERAL_REVISA, ADM_REVISA, COLABORADOR_REVISA, COLABORADOR_GABINETE, COLABORADOR_POLO	conforme escopo
GET	`/api/v1/persons/{id}`	`person.read`	mesmos acima e SELF quando aplicável	conforme escopo/ SELF
PATCH	`/api/v1/persons/{id}`	`person.update`	ADM_GERAL_REVISA, ADM_REVISA, COLABORADOR_REVISA, COLABORADOR_GABINETE, COLABORADOR_POLO	conforme escopo
POST	`/api/v1/persons/{id}/links`	`person.link`	ADM_GERAL_REVISA, ADM_REVISA, CHEFE_GABINETE, ADM_POLO	conforme escopo
GET	`/api/v1/consents`	`consent.read`	ADM_GERAL_REVISA, ADM_REVISA, ADM_POLO, COORDENADOR_POLO, CHEFE_GABINETE	conforme escopo
POST	`/api/v1/consents`	`consent.create`	ADM_GERAL_REVISA, ADM_REVISA, COLABORADOR_GABINETE, COLABORADOR_POLO, COLABORADOR_REVISA	conforme escopo

Método	Endpoint	Permissão	Perfis permitidos	Escopo
POST	`/api/v1/consents/{id}/revoke`	`consent.revoke`	ADM_GERAL_REVISA, ADM_REVISA, privacy team delegada	GLOBAL/REVISA

3.2.5 Captação territorial

Método	Endpoint	Permissão	Perfis permitidos	Escopo
POST	`/api/v1/contacts-capture`	`capture.create`	COLABORADOR_GABINETE, SUPERVISOR_EQUIPE_POLITICA, CHEFE_GABINETE, COLABORADOR_REVISA	GABINETE/EQUIPE/REVISA
GET	`/api/v1/contacts-capture`	`capture.read`	CHEFE_GABINETE, SUPERVISOR_EQUIPE_POLITICA, VEREADOR, ADM_REVISA, ADM_GERAL_REVISA	conforme escopo
GET	`/api/v1/contacts-capture/{id}`	`capture.read`	mesmos acima	conforme escopo
POST	`/api/v1/contacts-capture/{id}/classify`	`capture.classify`	CHEFE_GABINETE, SUPERVISOR_EQUIPE_POLITICA, ADM_REVISA	conforme escopo
POST	`/api/v1/contacts-capture/{id}/forward-to-polo`	`capture.forward`	CHEFE_GABINETE, ADM_REVISA, ADM_GERAL_REVISA	conforme escopo
POST	`/api/v1/contacts-capture/{id}/convert-beneficiary`	`capture.convert`	ADM_POLO, COORDENADOR_POLO, ADM_REVISA	conforme escopo

3.2.6 Polos e operação local

Método	Endpoint	Permissão	Perfis permitidos	Escopo
GET	`/api/v1/polos`	`polo.read`	todos perfis institucionais relevantes	conforme escopo
POST	`/api/v1/polos`	`polo.create`	ADM_GERAL_REVISA, ADM_REVISA	GLOBAL/ REVISA
GET	`/api/v1/polos/{id}`	`polo.read`	perfis vinculados	conforme escopo
PATCH	`/api/v1/polos/{id}`	`polo.update`	ADM_GERAL_REVISA, ADM_REVISA, ADM_POLO	conforme escopo
GET	`/api/v1/polos/{id}/beneficiarios`	`polo.manage_beneficiary` ou `person.read`	ADM_POLO, COORDENADOR_POLO, COLABORADOR_POLO, ADM_REVISA, ADM_GERAL_REVISA	POLO/ REVISA
POST	`/api/v1/polos/{id}/beneficiarios`	`polo.manage_beneficiary`	ADM_POLO, COORDENADOR_POLO, ADM_REVISA	POLO/ REVISA
GET	`/api/v1/polos/{id}/modalidades`	`modality.read`	perfis do polo e admins	POLO/ REVISA
POST	`/api/v1/polos/{id}/modalidades`	`modality.create`	ADM_POLO, COORDENADOR_POLO, ADM_REVISA	POLO/ REVISA
PATCH	`/api/v1/polos/{id}/modalidades/{modalidade_id}`	`modality.update`	ADM_POLO, COORDENADOR_POLO, ADM_REVISA	POLO/ REVISA
POST	`/api/v1/polos/{id}/frequencias`	`attendance.create`	COLABORADOR_POLO, COORDENADOR_POLO, ADM_POLO	POLO
GET	`/api/v1/polos/{id}/frequencias`	`attendance.read`	perfis do polo e admins	POLO/ REVISA
POST	`/api/v1/polos/{id}/ocorrencias`	`occurrence.create`	COLABORADOR_POLO, COORDENADOR_POLO, ADM_POLO	POLO

Método	Endpoint	Permissão	Perfis permitidos	Escopo
GET	`/api/v1/polos/{id}/ocorrencias`	`occurrence.read`	perfis do polo e admins	POLO/ REVISA
POST	`/api/v1/polos/{id}/daily-logs`	`daily_log.create`	COLABORADOR_POLO, COORDENADOR_POLO, ADM_POLO	POLO
GET	`/api/v1/polos/{id}/daily-logs`	`daily_log.read`	perfis do polo e admins	POLO/ REVISA
POST	`/api/v1/polos/{id}/purchase-requests`	`purchase_request.create`	COORDENADOR_POLO, ADM_POLO	POLO
GET	`/api/v1/polos/{id}/purchase-requests`	`purchase_request.read`	COORDENADOR_POLO, ADM_POLO, ADM_REVISA	POLO/ REVISA

3.2.7 Gabinete, ações e tarefas

Método	Endpoint	Permissão	Perfis permitidos	Escopo
GET	`/api/v1/gabinetes/{id}`	`cabinet.read`	CHEFE_GABINETE, SUPERVISOR_EQUIPE_POLITICA, VEREADOR, ADM_REVISA, ADM_GERAL_REVISA	GABINETE/ VEREADOR/ REVISA
GET	`/api/v1/gabinetes/{id}/acoes`	`cabinet.action.read`	CHEFE_GABINETE, SUPERVISOR_EQUIPE_POLITICA, VEREADOR, ADM_REVISA	conforme escopo
POST	`/api/v1/gabinetes/{id}/acoes`	`cabinet.action.create`	CHEFE_GABINETE, SUPERVISOR_EQUIPE_POLITICA	GABINETE
GET	`/api/v1/gabinetes/{id}/tarefas`	`task.read`	perfis do gabinete, vereador e admins	conforme escopo
POST	`/api/v1/gabinetes/{id}/tarefas`	`task.create`	CHEFE_GABINETE, SUPERVISOR_EQUIPE_POLITICA	GABINETE

Método	Endpoint	Permissão	Perfis permitidos	Escopo
PATCH	`/api/v1/tasks/{id}`	`task.update`	criador, superior hierárquico, admins	conforme escopo
POST	`/api/v1/tasks/{id}/complete`	`task.complete`	responsável, superior, admins	conforme escopo

3.2.8 Demandas

Método	Endpoint	Permissão	Perfis permitidos	Escopo
GET	`/api/v1/demands`	`demand.read`	perfis operacionais e admins	conforme escopo
POST	`/api/v1/demands`	`demand.create`	COLABORADOR_GABINETE, COLABORADOR_POLO, SUPERVISOR_EQUIPE_POLITICA, COORDENADOR_POLO, ADM_POLO, ADM_REVISA	conforme escopo
GET	`/api/v1/demands/{id}`	`demand.read`	perfis autorizados	conforme escopo
PATCH	`/api/v1/demands/{id}`	`demand.update`	responsável, superiores e admins	conforme escopo
POST	`/api/v1/demands/{id}/assign`	`demand.assign`	CHEFE_GABINETE, ADM_POLO, ADM_REVISA, ADM_GERAL_REVISA	conforme escopo
POST	`/api/v1/demands/{id}/close`	`demand.update`	responsável, superior, admins	conforme escopo

3.2.9 Eventos, atividades e agenda

Método	Endpoint	Permissão	Perfis permitidos	Escopo
GET	`/api/v1/events`	`event.read`	perfis institucionais + participantes autorizados	conforme escopo
POST	`/api/v1/events`	`event.create`	ADM_REVISA, CHEFE_GABINETE, ADM_POLO, COORDENADOR_POLO	conforme escopo

Método	Endpoint	Permissão	Perfis permitidos	Escopo
PATCH	`/api/v1/events/{id}`	`event.update`	criador, gestores do escopo, admins	conforme escopo
GET	`/api/v1/activities`	`event.read`	perfis do contexto	conforme escopo
POST	`/api/v1/activities`	`event.create`	ADM_POLO, COORDENADOR_POLO, CHEFE_GABINETE, ADM_REVISA	conforme escopo
POST	`/api/v1/activities/{id}/attendance`	`attendance.create`	COLABORADOR_POLO, COLABORADOR_GABINETE, COORDENADOR_POLO	conforme escopo

3.2.10 Dashboards, relatórios, geo, auditoria e privacidade

Método	Endpoint	Permissão	Perfis permitidos	Escopo
GET	`/api/v1/admin/dashboard`	`dashboard.admin.read`	ADM_GERAL_REVISA, ADM_REVISA	GLOBAL/ REVISA
GET	`/api/v1/vereadores/{id}/dashboard`	`dashboard.vereador.read`	VEREADOR dono, ADM_REVISA, ADM_GERAL_REVISA	VEREADOR/ REVISA
GET	`/api/v1/polos/{id}/dashboard`	`dashboard.polo.read`	ADM_POLO, COORDENADOR_POLO, ADM_REVISA	POLO/ REVISA
GET	`/api/v1/gabinetes/{id}/dashboard`	`dashboard.cabinet.read`	CHEFE_GABINETE, VEREADOR, ADM_REVISA	GABINETE/ VEREADOR/ REVISA
GET	`/api/v1/reports/*`	`report.read`	perfis gerenciais e admins	conforme escopo
POST	`/api/v1/reports/export`	`report.export`	perfis gerenciais e admins	conforme escopo
GET	`/api/v1/geo/layers`	`geo.read`	perfis autorizados	conforme escopo
POST	`/api/v1/geo/layers`	`geo.manage`	ADM_REVISA, ADM_GERAL_REVISA	REVISA/ GLOBAL

Método	Endpoint	Permissão	Perfis permitidos	Escopo
GET	`/api/v1/audit/logs`	`audit.read`	ADM_GERAL_REVISA, ADM_REVISA	GLOBAL/REVISA
GET	`/api/v1/privacy/requests`	`privacy.read`	ADM_GERAL_REVISA, ADM_REVISA	GLOBAL/REVISA
POST	`/api/v1/privacy/requests/{id}/process`	`privacy.process`	ADM_GERAL_REVISA, ADM_REVISA	GLOBAL/REVISA

---

4. DDL SQL final por schema

4.1 Schema `iam`

```sql
create schema if not exists iam;

create table iam.users (
    id uuid primary key,
    person_id uuid null,
    username varchar(120) not null unique,
    email varchar(180) not null unique,
    password_hash varchar(255) not null,
    status varchar(20) not null default 'ACTIVE',
    last_login_at timestamp null,
    must_reset_password boolean not null default false,
    created_at timestamp not null default now(),
    updated_at timestamp not null default now()
);

create table iam.roles (
    id uuid primary key,
    code varchar(80) not null unique,
    name varchar(120) not null,
    description text null,
    created_at timestamp not null default now()
);

create table iam.permissions (
    id uuid primary key,
    code varchar(120) not null unique,
    name varchar(120) not null,
    description text null,
    created_at timestamp not null default now()
);
```

```
create table iam.user_roles (
    id uuid primary key,
    user_id uuid not null references iam.users(id) on delete cascade,
    role_id uuid not null references iam.roles(id) on delete cascade,
    is_primary boolean not null default false,
    created_at timestamp not null default now(),
    unique (user_id, role_id)
);

create table iam.role_permissions (
    id uuid primary key,
    role_id uuid not null references iam.roles(id) on delete cascade,
    permission_id uuid not null references iam.permissions(id) on delete
cascade,
    created_at timestamp not null default now(),
    unique (role_id, permission_id)
);

create table iam.user_scope_assignments (
    id uuid primary key,
    user_id uuid not null references iam.users(id) on delete cascade,
    scope_type varchar(30) not null,
    scope_ref_id uuid not null,
    created_at timestamp not null default now(),
    unique (user_id, scope_type, scope_ref_id)
);

create table iam.refresh_tokens (
    id uuid primary key,
    user_id uuid not null references iam.users(id) on delete cascade,
    token_hash varchar(255) not null,
    expires_at timestamp not null,
    revoked_at timestamp null,
    created_at timestamp not null default now()
);
```

4.2 Schema `core`

```
create schema if not exists core;

create table core.organizations (
    id uuid primary key,
    type varchar(30) not null,
    name varchar(255) not null,
    legal_name varchar(255) null,
    document_number varchar(30) null,
    parent_organization_id uuid null references core.organizations(id),
    active boolean not null default true,
    created_at timestamp not null default now(),
```

```
    updated_at timestamp not null default now()
);

create table core.persons (
    id uuid primary key,
    full_name varchar(255) not null,
    social_name varchar(255) null,
    cpf varchar(20) null,
    birth_date date null,
    phone varchar(30) null,
    secondary_phone varchar(30) null,
    email varchar(180) null,
    gender varchar(30) null,
    notes text null,
    created_at timestamp not null default now(),
    updated_at timestamp not null default now()
);

create table core.addresses (
    id uuid primary key,
    person_id uuid null references core.persons(id) on delete cascade,
    organization_id uuid null references core.organizations(id) on delete
cascade,
    label varchar(60) null,
    street varchar(255) null,
    number varchar(30) null,
    complement varchar(120) null,
    district varchar(120) null,
    city varchar(120) null,
    state varchar(10) null,
    zip_code varchar(20) null,
    latitude numeric(10,7) null,
    longitude numeric(10,7) null,
    created_at timestamp not null default now(),
    check ((person_id is not null) <> (organization_id is not null))
);

create table core.vereadores (
    id uuid primary key,
    person_id uuid not null references core.persons(id),
    organization_id uuid not null references core.organizations(id),
    active boolean not null default true,
    created_at timestamp not null default now()
);

create table core.teams (
    id uuid primary key,
    organization_id uuid not null references core.organizations(id),
    name varchar(255) not null,
    team_type varchar(30) not null,
    active boolean not null default true,
```

```
    created_at timestamp not null default now()
);

create table core.team_members (
    id uuid primary key,
    team_id uuid not null references core.teams(id) on delete cascade,
    person_id uuid not null references core.persons(id),
    user_id uuid null references iam.users(id),
    function_name varchar(100) null,
    active boolean not null default true,
    created_at timestamp not null default now(),
    unique (team_id, person_id)
);

create table core.person_links (
    id uuid primary key,
    person_id uuid not null references core.persons(id),
    organization_id uuid null references core.organizations(id),
    vereador_id uuid null references core.vereadores(id),
    link_type varchar(50) not null,
    status varchar(30) not null default 'ACTIVE',
    start_date date null,
    end_date date null,
    metadata jsonb null,
    created_at timestamp not null default now()
);

create table core.consents (
    id uuid primary key,
    person_id uuid not null references core.persons(id),
    consent_type varchar(50) not null,
    granted boolean not null,
    version varchar(20) not null,
    granted_at timestamp null,
    revoked_at timestamp null,
    captured_by_user_id uuid null references iam.users(id),
    evidence_ref text null,
    created_at timestamp not null default now()
);

create table core.attachments (
    id uuid primary key,
    entity_type varchar(50) not null,
    entity_id uuid not null,
    file_name varchar(255) not null,
    mime_type varchar(100) null,
    storage_key text not null,
    uploaded_by_user_id uuid null references iam.users(id),
    created_at timestamp not null default now()
);
```

4.3 Schema `territory`

```sql
create schema if not exists territory;

create table territory.geo_entities (
    id uuid primary key,
    entity_type varchar(50) not null,
    entity_id uuid not null,
    latitude numeric(10,7) null,
    longitude numeric(10,7) null,
    geojson jsonb null,
    created_at timestamp not null default now(),
    unique (entity_type, entity_id)
);

create table territory.contacts_capture (
    id uuid primary key,
    captured_by_user_id uuid not null references iam.users(id),
    organization_id uuid null references core.organizations(id),
    vereador_id uuid null references core.vereadores(id),
    team_id uuid null references core.teams(id),
    person_id uuid null references core.persons(id),
    origin varchar(30) not null,
    classification varchar(30) not null,
    full_name varchar(255) not null,
    phone varchar(30) null,
    district varchar(120) null,
    notes text null,
    priority_level varchar(20) null,
    capture_status varchar(30) not null default 'NEW',
    latitude numeric(10,7) null,
    longitude numeric(10,7) null,
    created_at timestamp not null default now(),
    updated_at timestamp not null default now()
);

create table territory.leadership_signals (
    id uuid primary key,
    capture_id uuid null references territory.contacts_capture(id) on delete
cascade,
    person_id uuid null references core.persons(id),
    signal_type varchar(60) not null,
    role_name varchar(120) null,
    organization_name varchar(120) null,
    notes text null,
    created_by_user_id uuid not null references iam.users(id),
    created_at timestamp not null default now()
);

create table territory.demands (
```

```
    id uuid primary key,
    person_id uuid null references core.persons(id),
    capture_id uuid null references territory.contacts_capture(id),
    organization_id uuid null references core.organizations(id),
    vereador_id uuid null references core.vereadores(id),
    opened_by_user_id uuid not null references iam.users(id),
    assigned_to_user_id uuid null references iam.users(id),
    category varchar(60) not null,
    title varchar(255) not null,
    description text null,
    priority varchar(20) not null default 'MEDIUM',
    status varchar(20) not null default 'OPEN',
    due_at timestamp null,
    resolved_at timestamp null,
    resolution_notes text null,
    created_at timestamp not null default now(),
    updated_at timestamp not null default now()
);

create table territory.territorial_actions (
    id uuid primary key,
    organization_id uuid not null references core.organizations(id),
    vereador_id uuid null references core.vereadores(id),
    team_id uuid null references core.teams(id),
    created_by_user_id uuid not null references iam.users(id),
    title varchar(255) not null,
    description text null,
    action_type varchar(40) not null,
    priority varchar(20) not null default 'MEDIUM',
    status varchar(20) not null default 'OPEN',
    scheduled_at timestamp null,
    completed_at timestamp null,
    created_at timestamp not null default now()
);
```

4.4 Schema `polo`

```
create schema if not exists polo;

create table polo.units (
    id uuid primary key,
    organization_id uuid not null unique references core.organizations(id),
    code varchar(50) null,
    address_label varchar(255) null,
    active boolean not null default true,
    created_at timestamp not null default now()
);

create table polo.beneficiarios (
    id uuid primary key,
```

```
    polo_id uuid not null references polo.units(id),
    person_id uuid not null references core.persons(id),
    source_capture_id uuid null references territory.contacts_capture(id),
    status varchar(30) not null default 'PRE_CADASTRADO',
    admitted_at timestamp null,
    discharged_at timestamp null,
    created_at timestamp not null default now(),
    unique (polo_id, person_id)
);

create table polo.modalidades (
    id uuid primary key,
    polo_id uuid not null references polo.units(id),
    name varchar(255) not null,
    description text null,
    active boolean not null default true,
    created_at timestamp not null default now()
);

create table polo.matriculas_modalidade (
    id uuid primary key,
    beneficiario_id uuid not null references polo.beneficiarios(id) on
delete cascade,
    modalidade_id uuid not null references polo.modalidades(id),
    status varchar(30) not null default 'ATIVA',
    start_date date not null,
    end_date date null,
    created_at timestamp not null default now()
);

create table polo.frequencias (
    id uuid primary key,
    beneficiario_id uuid not null references polo.beneficiarios(id) on
delete cascade,
    modalidade_id uuid null references polo.modalidades(id),
    registered_by_user_id uuid not null references iam.users(id),
    activity_date date not null,
    present boolean not null,
    notes text null,
    created_at timestamp not null default now(),
    unique (beneficiario_id, modalidade_id, activity_date)
);

create table polo.ocorrencias (
    id uuid primary key,
    polo_id uuid not null references polo.units(id),
    beneficiario_id uuid null references polo.beneficiarios(id),
    registered_by_user_id uuid not null references iam.users(id),
    severity varchar(20) not null,
    title varchar(255) not null,
    description text not null,
```

```
    status varchar(20) not null default 'OPEN',
    created_at timestamp not null default now(),
    updated_at timestamp not null default now()
);

create table polo.daily_logs (
    id uuid primary key,
    polo_id uuid not null references polo.units(id),
    created_by_user_id uuid not null references iam.users(id),
    log_date date not null,
    content text not null,
    created_at timestamp not null default now(),
    unique (polo_id, log_date, created_by_user_id)
);

create table polo.purchase_requests (
    id uuid primary key,
    polo_id uuid not null references polo.units(id),
    requested_by_user_id uuid not null references iam.users(id),
    title varchar(255) not null,
    description text null,
    status varchar(20) not null default 'OPEN',
    priority varchar(20) not null default 'MEDIUM',
    created_at timestamp not null default now(),
    updated_at timestamp not null default now()
);
```

4.5 Schema `events`

```
create schema if not exists events;

create table events.events (
    id uuid primary key,
    organization_id uuid null references core.organizations(id),
    vereador_id uuid null references core.vereadores(id),
    created_by_user_id uuid not null references iam.users(id),
    title varchar(255) not null,
    description text null,
    event_type varchar(40) not null,
    status varchar(20) not null default 'PLANNED',
    start_at timestamp not null,
    end_at timestamp null,
    created_at timestamp not null default now(),
    updated_at timestamp not null default now()
);

create table events.activities (
    id uuid primary key,
    event_id uuid null references events.events(id) on delete cascade,
    polo_id uuid null references polo.units(id),
```

```
    organization_id uuid null references core.organizations(id),
    title varchar(255) not null,
    activity_type varchar(40) not null,
    starts_at timestamp not null,
    ends_at timestamp null,
    recurrence_rule varchar(255) null,
    created_by_user_id uuid not null references iam.users(id),
    created_at timestamp not null default now()
);

create table events.participations (
    id uuid primary key,
    activity_id uuid not null references events.activities(id) on delete
cascade,
    person_id uuid not null references core.persons(id),
    registered_by_user_id uuid not null references iam.users(id),
    checkin_at timestamp null,
    status varchar(20) not null default 'REGISTERED',
    notes text null,
    created_at timestamp not null default now(),
    unique (activity_id, person_id)
);
```

4.6 Schema `workflow`

```
create schema if not exists workflow;

create table workflow.tasks (
    id uuid primary key,
    organization_id uuid null references core.organizations(id),
    vereador_id uuid null references core.vereadores(id),
    polo_id uuid null references polo.units(id),
    person_id uuid null references core.persons(id),
    demand_id uuid null references territory.demands(id),
    assigned_to_user_id uuid null references iam.users(id),
    created_by_user_id uuid not null references iam.users(id),
    task_type varchar(50) not null,
    title varchar(255) not null,
    description text null,
    priority varchar(20) not null default 'MEDIUM',
    status varchar(20) not null default 'OPEN',
    due_at timestamp null,
    completed_at timestamp null,
    created_at timestamp not null default now(),
    updated_at timestamp not null default now()
);

create table workflow.task_outcomes (
    id uuid primary key,
    task_id uuid not null references workflow.tasks(id) on delete cascade,
```

```
    outcome_type varchar(50) not null,
    notes text null,
    recorded_by_user_id uuid not null references iam.users(id),
    recorded_at timestamp not null default now()
);
```

4.7 Schema `governance`

```
create schema if not exists governance;

create table governance.audit_logs (
    id bigserial primary key,
    user_id uuid null references iam.users(id),
    action varchar(120) not null,
    entity_schema varchar(60) not null,
    entity_name varchar(120) not null,
    entity_id uuid null,
    old_values_json jsonb null,
    new_values_json jsonb null,
    ip_address inet null,
    user_agent text null,
    created_at timestamp not null default now()
);

create table governance.access_logs (
    id bigserial primary key,
    user_id uuid null references iam.users(id),
    event_type varchar(60) not null,
    success boolean not null,
    ip_address inet null,
    user_agent text null,
    created_at timestamp not null default now()
);

create table governance.export_logs (
    id bigserial primary key,
    user_id uuid not null references iam.users(id),
    export_type varchar(60) not null,
    filter_json jsonb null,
    row_count integer null,
    created_at timestamp not null default now()
);

create table governance.privacy_requests (
    id uuid primary key,
    person_id uuid not null references core.persons(id),
    request_type varchar(30) not null,
    status varchar(20) not null default 'OPEN',
    requested_at timestamp not null default now(),
    processed_at timestamp null,
```

```
    processed_by_user_id uuid null references iam.users(id),
    notes text null
);
```

4.8 Schema `analytics`

```
create schema if not exists analytics;

create table analytics.feature_store_operational (
    id uuid primary key,
    person_id uuid not null references core.persons(id),
    ref_date date not null,
    interactions_30d integer not null default 0,
    activities_90d integer not null default 0,
    open_demands_count integer not null default 0,
    active_beneficiary boolean not null default false,
    linked_vereador_id uuid null references core.vereadores(id),
    linked_polo_id uuid null references polo.units(id),
    created_at timestamp not null default now(),
    unique (person_id, ref_date)
);

create materialized view analytics.mv_vereador_dashboard as
select
    v.id as vereador_id,
    count(distinct cc.id) as total_captures,
    count(distinct pb.id) as total_beneficiarios,
    count(distinct d.id) filter (where d.status = 'OPEN') as open_demands,
    count(distinct t.id) filter (where t.status = 'OPEN') as open_tasks
from core.vereadores v
left join territory.contacts_capture cc on cc.vereador_id = v.id
left join territory.demands d on d.vereador_id = v.id
left join workflow.tasks t on t.vereador_id = v.id
left join territory.contacts_capture cc2 on cc2.vereador_id = v.id
left join polo.beneficiarios pb on pb.source_capture_id = cc2.id
group by v.id;
```

---

5. Contratos OpenAPI por módulo

5.1 Convenções globais do contrato

Base

- versão: `v1`
- auth: `Bearer JWT`
- paginação padrão: `page`, `page_size`
- ordenação padrão: `sort_by`, `sort_order`

- filtro por período: `date_from`, `date_to`

Envelope padrão de erro

```
ErrorResponse:
  type: object
  required: [detail]
  properties:
    detail:
      type: string
    code:
      type: string
      nullable: true
    fields:
      type: object
      additionalProperties: true
      nullable: true
```

Envelope padrão de paginação

```
PaginatedMeta:
  type: object
  required: [page, page_size, total]
  properties:
    page:
      type: integer
    page_size:
      type: integer
    total:
      type: integer
```

5.2 Módulo Auth

```
/auth/login:
  post:
    summary: Login do usuário
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required: [username, password]
            properties:
              username:
                type: string
```

```
              password:
                type: string
    responses:
      '200':
        description: Token emitido
        content:
          application/json:
            schema:
              type: object
              required: [access_token, refresh_token, token_type]
              properties:
                access_token:
                  type: string
                refresh_token:
                  type: string
                token_type:
                  type: string
                  example: Bearer
                expires_in:
                  type: integer

/auth/me:
  get:
    summary: Retorna usuário autenticado
    security:
      - bearerAuth: []
    responses:
      '200':
        description: Perfil do usuário
```

5.3 Módulo Users

```
/components/schemas/User:
  type: object
  required: [id, username, email, status]
  properties:
    id:
      type: string
      format: uuid
    username:
      type: string
    email:
      type: string
      format: email
    status:
      type: string
    roles:
      type: array
      items:
        type: string
```

```
/users:
  get:
    summary: Lista usuários
    security:
      - bearerAuth: []
    parameters:
      - in: query
        name: page
        schema: { type: integer, default: 1 }
      - in: query
        name: page_size
        schema: { type: integer, default: 20 }
    responses:
      '200':
        description: Lista paginada
  post:
    summary: Cria usuário
    security:
      - bearerAuth: []
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required: [username, email, password]
            properties:
              username: { type: string }
              email: { type: string, format: email }
              password: { type: string }
              person_id: { type: string, format: uuid, nullable: true }
```

5.4 Módulo Persons

```
/components/schemas/Person:
  type: object
  required: [id, full_name]
  properties:
    id:
      type: string
      format: uuid
    full_name:
      type: string
    social_name:
      type: string
      nullable: true
    cpf:
      type: string
      nullable: true
```

```
    birth_date:
      type: string
      format: date
      nullable: true
    phone:
      type: string
      nullable: true
    email:
      type: string
      nullable: true

/persons:
  post:
    summary: Cria pessoa
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required: [full_name]
            properties:
              full_name: { type: string }
              social_name: { type: string, nullable: true }
              cpf: { type: string, nullable: true }
              birth_date: { type: string, format: date, nullable: true }
              phone: { type: string, nullable: true }
              secondary_phone: { type: string, nullable: true }
              email: { type: string, format: email, nullable: true }
```

5.5 Módulo Contacts Capture

```
/components/schemas/ContactCapture:
  type: object
  required: [id, origin, classification, full_name, capture_status]
  properties:
    id:
      type: string
      format: uuid
    origin:
      type: string
      enum: [MOBILE, WEB, PUBLIC]
    classification:
      type: string
      enum: [BENEFICIARIO, APOIADOR, LIDERANCA, CIDADAO]
    full_name:
      type: string
    phone:
      type: string
      nullable: true
```

```
    district:
      type: string
      nullable: true
    notes:
      type: string
      nullable: true
    capture_status:
      type: string
      enum: [NEW, TRIAGED, FORWARDED, CONVERTED, ARCHIVED]

/contacts-capture:
  post:
    summary: Cria captação territorial
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required: [origin, classification, full_name]
            properties:
              origin: { type: string }
              classification: { type: string }
              full_name: { type: string }
              phone: { type: string, nullable: true }
              district: { type: string, nullable: true }
              notes: { type: string, nullable: true }
              vereador_id: { type: string, format: uuid, nullable: true }
              team_id: { type: string, format: uuid, nullable: true }
              latitude: { type: number, nullable: true }
              longitude: { type: number, nullable: true }

/contacts-capture/{id}/classify:
  post:
    summary: Reclassifica captação
    parameters:
      - in: path
        name: id
        required: true
        schema: { type: string, format: uuid }
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required: [classification]
            properties:
              classification: { type: string }
              notes: { type: string, nullable: true }
```

5.6 Módulo Polos

```
/components/schemas/PoloBeneficiary:
  type: object
  required: [id, polo_id, person_id, status]
  properties:
    id:
      type: string
      format: uuid
    polo_id:
      type: string
      format: uuid
    person_id:
      type: string
      format: uuid
    status:
      type: string
      enum: [PRE_CADASTRADO, ATIVO, INATIVO, ENCERRADO]

/polos/{id}/beneficiarios:
  get:
    summary: Lista beneficiários do polo
  post:
    summary: Vincula beneficiário ao polo
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required: [person_id]
            properties:
              person_id: { type: string, format: uuid }
              source_capture_id: { type: string, format: uuid, nullable:
true }
              status: { type: string, default: PRE_CADASTRADO }

/polos/{id}/frequencias:
  post:
    summary: Registra frequência
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required: [beneficiario_id, activity_date, present]
            properties:
              beneficiario_id: { type: string, format: uuid }
              modalidade_id: { type: string, format: uuid, nullable: true }
```

```
              activity_date: { type: string, format: date }
              present: { type: boolean }
              notes: { type: string, nullable: true }
```

5.7 Módulo Demands

```
/components/schemas/Demand:
  type: object
  required: [id, category, title, priority, status]
  properties:
    id:
      type: string
      format: uuid
    category:
      type: string
    title:
      type: string
    description:
      type: string
      nullable: true
    priority:
      type: string
      enum: [LOW, MEDIUM, HIGH, URGENT]
    status:
      type: string
      enum: [OPEN, IN_PROGRESS, BLOCKED, RESOLVED, CLOSED]

/demands:
  post:
    summary: Cria demanda
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required: [category, title]
            properties:
              person_id: { type: string, format: uuid, nullable: true }
              capture_id: { type: string, format: uuid, nullable: true }
              organization_id: { type: string, format: uuid, nullable: true }
              vereador_id: { type: string, format: uuid, nullable: true }
              category: { type: string }
              title: { type: string }
              description: { type: string, nullable: true }
              priority: { type: string, default: MEDIUM }
              due_at: { type: string, format: date-time, nullable: true }
```

5.8 Módulo Tasks

```
/components/schemas/Task:
  type: object
  required: [id, task_type, title, priority, status]
  properties:
    id:
      type: string
      format: uuid
    task_type:
      type: string
    title:
      type: string
    description:
      type: string
      nullable: true
    priority:
      type: string
      enum: [LOW, MEDIUM, HIGH, URGENT]
    status:
      type: string
      enum: [OPEN, IN_PROGRESS, COMPLETED, CANCELLED]

/tasks:
  post:
    summary: Cria tarefa
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required: [task_type, title]
            properties:
              organization_id: { type: string, format: uuid, nullable: true }
              vereador_id: { type: string, format: uuid, nullable: true }
              polo_id: { type: string, format: uuid, nullable: true }
              person_id: { type: string, format: uuid, nullable: true }
              demand_id: { type: string, format: uuid, nullable: true }
              assigned_to_user_id: { type: string, format: uuid, nullable:
true }
              task_type: { type: string }
              title: { type: string }
              description: { type: string, nullable: true }
              priority: { type: string, default: MEDIUM }
              due_at: { type: string, format: date-time, nullable: true }
```

5.9 Módulo Dashboards

```
/vereadores/{id}/dashboard:
  get:
    summary: Dashboard executivo do vereador
    parameters:
      - in: path
        name: id
        required: true
        schema: { type: string, format: uuid }
    responses:
      '200':
        description: Indicadores do vereador
        content:
          application/json:
            schema:
              type: object
              properties:
                vereador_id: { type: string, format: uuid }
                total_captures: { type: integer }
                total_beneficiarios: { type: integer }
                open_demands: { type: integer }
                open_tasks: { type: integer }
                poles:
                  type: array
                  items:
                    type: object
                    properties:
                      polo_id: { type: string, format: uuid }
                      name: { type: string }
                      active_beneficiaries: { type: integer }
```

---

6. Estrutura exata de pastas do monorepo

6.1 Visão final

```
revisa-platform/
├── README.md
├── .editorconfig
├── .gitignore
├── .env.example
├── docker-compose.yml
├── Makefile
├── docs/
│   ├── architecture/
│   │   ├── execution-spec.md
```

```
│   │   ├── permissions-matrix.md
│   │   ├── database-ddl.md
│   │   └── api-contracts.md
│   ├── business/
│   ├── governance/
│   └── runbooks/
├── apps/
│   ├── api/
│   ├── web/
│   └── mobile/
├── packages/
│   ├── domain-contracts/
│   ├── ui-tokens/
│   ├── enums/
│   └── lint-config/
├── database/
│   ├── ddl/
│   ├── seeds/
│   ├── views/
│   └── fixtures/
├── infra/
│   ├── docker/
│   ├── nginx/
│   ├── ci/
│   ├── monitoring/
│   └── terraform/
└── scripts/
```

6.2 Backend `apps/api`

```
apps/api/
├── pyproject.toml
├── alembic.ini
├── Dockerfile
├── app/
│   ├── main.py
│   ├── core/
│   │   ├── config.py
│   │   ├── database.py
│   │   ├── security.py
│   │   ├── permissions.py
│   │   ├── scope.py
│   │   ├── exceptions.py
│   │   ├── logging.py
│   │   └── telemetry.py
│   ├── api/
│   │   ├── deps/
```

```
│   │   │   ├── auth.py
│   │   │   ├── permissions.py
│   │   │   └── scope.py
│   │   └── v1/
│   │       ├── routers/
│   │       │   ├── auth.py
│   │       │   ├── users.py
│   │       │   ├── organizations.py
│   │       │   ├── vereadores.py
│   │       │   ├── teams.py
│   │       │   ├── persons.py
│   │       │   ├── consents.py
│   │       │   ├── contacts_capture.py
│   │       │   ├── polos.py
│   │       │   ├── polo_beneficiaries.py
│   │       │   ├── modalities.py
│   │       │   ├── attendances.py
│   │       │   ├── occurrences.py
│   │       │   ├── daily_logs.py
│   │       │   ├── purchase_requests.py
│   │       │   ├── demands.py
│   │       │   ├── gabinete_actions.py
│   │       │   ├── tasks.py
│   │       │   ├── events.py
│   │       │   ├── activities.py
│   │       │   ├── dashboards.py
│   │       │   ├── reports.py
│   │       │   ├── geo.py
│   │       │   ├── audit.py
│   │       │   └── privacy.py
│   │       └── api.py
│   ├── domain/
│   │   ├── iam/
│   │   │   ├── models.py
│   │   │   ├── schemas.py
│   │   │   ├── repository.py
│   │   │   ├── service.py
│   │   │   └── policy.py
│   │   ├── core/
│   │   ├── territory/
│   │   ├── polo/
│   │   ├── events/
│   │   ├── workflow/
│   │   ├── governance/
│   │   └── analytics/
│   ├── shared/
│   │   ├── pagination.py
│   │   ├── filters.py
│   │   ├── enums.py
│   │   └── audit.py
```

```
│   └── tests/
│       ├── unit/
│       ├── integration/
│       └── contract/
├── alembic/
│   ├── env.py
│   └── versions/
└── scripts/
```

Responsabilidade por pasta

- `core/`: infraestrutura técnica transversal
- `api/deps/`: dependências de segurança, permissão, injeção e escopo
- `api/v1/routers/`: camada HTTP pura, sem regra de negócio complexa
- `domain/*/models.py`: modelos ORM do domínio
- `domain/*/schemas.py`: DTOs Pydantic do domínio
- `domain/*/repository.py`: acesso a dados
- `domain/*/service.py`: regra de negócio e orquestração
- `domain/*/policy.py`: regra de autorização fina do domínio
- `shared/`: elementos genéricos sem dependência circular

Convenções obrigatórias do backend

- router não acessa ORM diretamente
- service não conhece HTTP
- repository não aplica regra de negócio
- policy concentra checagem contextual fina do módulo
- toda mutação chama auditoria

6.3 Frontend Web `apps/web`

```
apps/web/
├── package.json
├── next.config.mjs
├── tsconfig.json
├── Dockerfile
├── src/
│   ├── app/
│   │   ├── (public)/
│   │   ├── (auth)/
│   │   ├── (admin)/
│   │   │   ├── users/
│   │   │   ├── organizations/
│   │   │   ├── vereadores/
│   │   │   ├── audit/
│   │   │   └── privacy/
│   │   ├── (polo)/
│   │   │   ├── dashboard/
```

```
│   │   │   ├── beneficiarios/
│   │   │   ├── modalidades/
│   │   │   ├── frequencias/
│   │   │   ├── ocorrencias/
│   │   │   ├── diario/
│   │   │   └── compras/
│   │   ├── (gabinete)/
│   │   │   ├── dashboard/
│   │   │   ├── captacoes/
│   │   │   ├── tarefas/
│   │   │   ├── demandas/
│   │   │   └── acoes/
│   │   ├── (vereador)/
│   │   │   ├── dashboard/
│   │   │   ├── polos/
│   │   │   ├── relatorios/
│   │   │   └── mapa/
│   │   └── layout.tsx
│   ├── features/
│   │   ├── auth/
│   │   ├── users/
│   │   ├── organizations/
│   │   ├── persons/
│   │   ├── contacts-capture/
│   │   ├── polo/
│   │   ├── gabinete/
│   │   ├── vereador/
│   │   ├── events/
│   │   ├── workflow/
│   │   ├── reports/
│   │   └── governance/
│   ├── components/
│   │   ├── ui/
│   │   ├── charts/
│   │   ├── maps/
│   │   └── forms/
│   ├── lib/
│   │   ├── api-client.ts
│   │   ├── auth.ts
│   │   ├── permissions.ts
│   │   └── formatters.ts
│   └── types/
└── tests/
```

Responsabilidade por pasta

- `app/`: roteamento e composition final da tela
- `features/`: lógica de caso de uso por domínio
- `components/ui/`: componentes genéricos reutilizáveis

- `components/forms/`: campos e formulários compartilhados
- `lib/permissions.ts`: checagem client-side complementar de visibilidade
- `types/`: tipos derivados dos contratos compartilhados

Convenções obrigatórias do web

- tela não chama fetch bruto fora de `lib/api-client.ts`
- formulário por domínio em `features/<domínio>/components`
- guardas de rota por perfil e escopo
- nenhuma regra crítica de segurança fica só no frontend

6.4 Mobile `apps/mobile`

```
apps/mobile/
├── pubspec.yaml
├── lib/
│   ├── main.dart
│   ├── app/
│   │   ├── router.dart
│   │   ├── theme.dart
│   │   └── bootstrap.dart
│   ├── core/
│   │   ├── api_client.dart
│   │   ├── auth_store.dart
│   │   ├── local_db.dart
│   │   ├── sync_engine.dart
│   │   └── validators.dart
│   ├── features/
│   │   ├── auth/
│   │   ├── captura/
│   │   │   ├── pages/
│   │   │   ├── widgets/
│   │   │   ├── models/
│   │   │   └── service.dart
│   │   ├── demandas/
│   │   ├── agenda/
│   │   ├── contatos/
│   │   └── sync/
│   ├── shared/
│   │   ├── widgets/
│   │   ├── enums/
│   │   └── formatters/
│   └── generated/
└── test/
```

Responsabilidade do mobile

- autenticação

- captação territorial
- fila de sincronização
- consulta operacional mínima do próprio contexto
- coleta de consentimento e geolocalização quando autorizado

Convenções obrigatórias do mobile

- sync desacoplado do formulário
- persistência local para fila de envio
- payloads compatíveis com `packages/domain-contracts`
- nenhuma lógica de permissão definitiva implementada apenas no app

---

6.5 Pacotes compartilhados `packages/`

```
packages/
├── domain-contracts/
│   ├── openapi/
│   ├── json-schemas/
│   └── README.md
├── enums/
│   ├── role-codes.ts
│   ├── scope-types.ts
│   ├── demand-status.ts
│   └── capture-classification.ts
├── ui-tokens/
│   ├── colors.ts
│   ├── spacing.ts
│   └── typography.ts
└── lint-config/
```

Responsabilidade

- `domain-contracts/`: contrato canônico entre API, web e mobile
- `enums/`: enumerações estáveis compartilhadas
- `ui-tokens/`: consistência visual multi-app
- `lint-config/`: padronização de código

---

7. Convenções finais de implementação por módulo

7.1 Sequência recomendada

1. `iam`
2. `core`
3. `territory`
4. `polo`
5. `workflow`

6. `events`
7. `governance`
8. `analytics`

7.2 Regra de entrega por PR

Cada PR deve conter no máximo:

- 1 módulo principal
- alterações necessárias em pacotes compartilhados
- migrations correspondentes
- seeds quando houver
- testes do módulo

Não misturar no mesmo PR:

- IAM + Polo + Dashboard + Geo

7.3 Definition of Done

Um módulo só entra como concluído quando possuir:

- DDL aplicada
- seed mínima
- endpoints documentados
- matriz de permissões aplicada
- testes de autenticação/autorização
- logs essenciais
- tela operacional ou integração de consumo

---

8. Fechamento executivo

Esta especificação de execução fixa o desenho final do sistema em termos implementáveis.

Ela resolve quatro problemas críticos:

- transforma perfis abstratos em autorização concreta por endpoint
- transforma arquitetura em schemas SQL executáveis
- transforma domínio em contratos OpenAPI por módulo
- transforma ideia de repositório em estrutura final de monorepo pronta para desenvolvimento disciplinado

A partir daqui, o caminho correto de produção é gerar os artefatos operacionais derivados:

- migrations reais por schema
- arquivo OpenAPI consolidado
- seeds de papéis/permissões
- backlog técnico por PR
- runbook de deploy e rollback